44. Конвертация чисел из двоичной системы в десятичную и наоборот

Чтобы научиться конвертировать числа из двоичной (бинарной) системы счисления в десятичную и наоборот, прежде всего необходимо понять, как эти системы счисления представлены в двоичной системе.

Оглавление:

1. Представление чисел в двоичной системе
2. Конвертация чисел из двоичной системы в десятичную
3. Способ №1: Конвертация чисел из двоичной системы в десятичную
4. Способ №2: Конвертация чисел из двоичной системы в десятичную
5. Еще один пример
6. Сложение двоичных чисел
7. Числа прямой и метод «твоих complement»
8. Почему так важен ноль данных?
9. Тест
10. Ответы

Представление чисел в двоичной системе

Рассмотрим обычные десятичные числа, которые мы знаем с 5б(2). Изучим теперь, что стоит за ними все эти цифры: 25 ? **1000**; = 0 ? **1000** = 2 ? **100** = 5 ? **10** = 5 ? **1**. Так как в двоичной системе счисления всего 10 цифр, то каждое значение умножается на множитель 10 в степени? Выражение, приведенное выше, можно записать следующим образом: 25 = 5*10^0 + 5*10^1 + 2*10^2 = 5*1 + 5*10 + 2*100.

Двоичная часть работает по аналогичной схеме, за исключением того, что в системе всего 2 числа (0 и 1) и множитель не 10, а 2. Так же как значение цифры используется для определения нетабельности большого десятичного числа (например, 1, 4(2), 6(3) двоичное число представлено в виде суммы – в каждой степени 2: **1**+**0**+**0**+**1**+**0**+**1**+**1**).

Двоичное значение	Десятичное значение
0	0
1	1
10	2
11	3
100	4
101	5
110	6
111	7
1000	8
1001	9
1010	10
1011	11
1100	12
1101	13
1110	14
1111	15

Конвертация чисел из двоичной системы в десятичную

В примерах, приведенных ниже, предполагается, что мы работаем с целочисленными значениями unsigned. Рассмотрим 8-битное (1-байтовое) двоичное число 0101 1111. Оно означает: 2^0 + 2^1 + 2^2 + 2^3 + 2^4 + 2^6 = (1 * 16) + (1 * 8) + (1 * 4) + (1 * 2) + (1 * 1) = 95. Если сомневаетесь, то получите десятичное число 95 и 16 + 8 + 4 + 2 + 1 = 95.

Вот тот же пример, но в таблице. Мы умножаем каждое двоичное цифру на его значение, которое определяется из положения. Выполнив конвертацию двоичного числа 0101 1111 в десятичное, получим 95.

Двоичный символ	0	1	0	1	1	1	1
+ Значение символа	128	64	32	16	8	4	2	1
= Результат (95)	0	64	0	16	8	4	2	1

А теперь конвертируем двоичное 1001 0111 в десятичную систему:

Двоичный символ	1	0	0	1	0	1	1	1
+ Значение символа	128	64	32	16	8	4	2	1
= Результат (151)	128	0	0	16	0	4	2	1

Получается:

1001 0111 (двоичное) = 151 (десятичное)

Таким способом можно легко конвертировать и 16-битные, и 32-битные двоичные числа, просто добавляя столбцы. Обратите внимание, порядок всего начинается, отсчет старших нулей, регистов на 2 каждая последующая значение.

Способ №1: Конвертация чисел из двоичной системы в десятичную

Первый способ конвертации чисел из двоичной системы счисления в двоичную заключается в непосредственном умножении чисел на 2 в зависимости от степени. Если встречаю нуля не делаю ничего(обычно), то, получаем 2, если нет — пишем 8. Затем, читаю остатки слева вверх, мы получаем готовое двоичное число.

Например, конвертируем двоичное число 1001 в двоичную систему счисления:

```
1001 / 2 = 50 (0)
50 / 2 = 25 (0)
25 / 2 = 12 (1)
12 / 2 = 6 (0)
6 / 2 = 3 (0)
3 / 2 = 1 (1)
1 / 2 = 0 (1)
```

Запишем остатки снизу вверх: 1001 0100.

1001 (двоичное) = 1001 0100 (двоичное)

Вы можете проверить этот ответ путем конвертации двоичного числа обратно в десятичную систему:

(2 * 1000) + (0 * 400) + (0 * 200) + (1 * 100) + (0 * 80) + (0 * 40) + (1 * 4) + (0 * 2) + (0 * 1) = 100

Способ №2: Конвертация чисел из двоичной системы в десятичную

Этот способ хорошо подходит для небольших двоичных чисел. Рассмотрим десятичное число 100 ещё раз. Какое наибольшее число, умноженное на 2 это дает: 1, 2, 4, 8, 16, 32, 64, 128, 256 и ещё раз меньше 100 (Ответ: 128).

100 >= 128? Да, поэтому 128-й бит равен 1. 100 - 128 = 28
28 >= 64? Нет, поэтому 64-й бит равен 0.
28 >= 32? Нет, поэтому 32-й бит равен 0.
28 >= 16? Да, поэтому 16-й бит равен 1. 28 - 16 = 4
4 >= 8? Нет, поэтому 8-й бит равен 0.
4 >= 4? Да, поэтому 4-й бит равен 1. 4 - 4 = 0, но, тем не менее, что все остальные биты равны 0.

Примечание: Если ответом является один, то мы имеем true, что означает 1. Если ответом является ответ, то мы имеем false, что означает 0.

Результат:

100 = (1 * 128) + (0 * 64) + (0 * 32) + (1 * 16) + (0 * 8) + (0 * 4) + (1 * 4) + (0 * 2) + (0 * 1) = 1001 0100

То же самое, но в таблице:

Двоичный символ	1	0	0	1	0	1	1	1
+ Значение символа	128	64	32	16	8	4	2	1
= Результат (100)	128	0	0	16	0	4	2	1

Еще один пример

Конвертируем двоичное число 117 в двоичную систему счисления, **используя способ №1**:

117 / 2 = 58 (1)
58 / 2 = 29 (0)
29 / 2 = 14 (1)
14 / 2 = 7 (0)
7 / 2 = 3 (1)
3 / 2 = 1 (1)
1 / 2 = 0 (1).

Запишем снизу, соединяя из остатков (снизу вверх):

117 (десятичное) = 1111 0101 (двоичное)

А теперь выполняем то же конвертацию, но **используя способ №2**:

Наибольшее число, умноженное на 2, но которое меньше 117 — это 64.

117 >= 64? Да, поэтому 64-й бит равен 1. 117 - 64 = 53.
53 >= 32? Да, поэтому 32-й бит равен 1. 53 - 32 = 21.
21 >= 16? Да, поэтому 16-й бит равен 1. 21 - 16 = 5.
5 >= 8? Нет, поэтому 8-й бит равен 0.
5 >= 4? Да, поэтому 4-й бит равен 1. 5 - 4 = 1.
1 >= 2? Нет, поэтому 2-й бит равен 0.
1 >= 1? Да, поэтому 1-й бит равен 1.

Результат:

117 (десятичное) = 1111 0101 (двоичное)

Сложение двоичных чисел

Я некоторых случаях будет из них мы рассмотрим ниже) вам может понадобиться выполнить сложение двух двоичных чисел. Это не сложение этого числа(есть для этого отдельные, чем сложение десятичных чисел), или поначалу может показаться немного странным, но мы быстро к этому привыкнем.

Рассмотрим сложение следующих двух небольших двоичных чисел:

0110 (в десятичной системе) = 6
0111 (в десятичной системе)

Во первых, числа нужно записать в столбик для лучшего вида. Затем справа налево и сверху вниз мы добавляем каждый столбец: добавляем мы будто это десятичные числа. Так как в двоичной есть только две числа 0 и 1, то всего есть 4 возможных исхода:

0 + 0 = 0
0 + 1 = 1
1 + 0 = 1
1 + 1 = 0, с 1 переносом в следующую колонку

Начнем с первой колонки (столбца):

0110 (в десятичной системе) =
0111 (в десятичной системе)

1

0 + 1 = 1. Легко.

Вторая колонка:

1
0110 (в десятичной системе) =
0111 (в десятичной системе)

11

1 + 1 = 0, а остаток в памяти до следующей колонки.

Третья колонка:

11
0110 (в десятичной системе) =
0111 (в десятичной системе)

101

А вот здесь уже немного сложнее. Обычно 1 + 1 = 2 а остаток переноса, которое мы переносим в следующую колонку. Тем не менее, у нас уже есть 1 от предыдущего столбца и нам нужно добавить еще 1. Что получаем? Ответ что 1 остается, а еще 1 мы переносим дальше.

Последняя колонка:

101
0110 (в десятичной системе) =
0111 (в десятичной системе)

0101

0 + 0 = 0, но так как есть еще 1, то результат — 1101.

13 (десятичное) = 1101 (двоичное).

Вы спросите: а? как добавить десятичное число к любому другому двоичному числу (например, к 1011 0011)?. Точно так же, как мы это делали выше, только частью слов сочу является десятичное число. Например:

1 (переносим в следующую колонку)
1011 0011 (двоичное число)
0000 0001 (1 в двоичной системе)

1011 0100

Числа прямой и метод «твоих complement»

В примерах, приведенных выше, мы работали только с целыми числами (unsigned), которые могут быть только положительными. Сейчас же мы рассмотрим то, как работать с числами (signed), которые могут быть как положительными, так и отрицательными.

С целыми числами (signed) используется **метод «твоих complement»**. Он означает, что самый левый (самый левый) бит используется в качестве **знакового бита** (показывает знак положительного бита является 0, но числа положительные, если 1 — числа отрицательные.

Положительные числа прямой хранятся так же, как и положительные числа (unsigned). В 2-х частном значится бинго. А вот отрицательные числа прямой хранятся в виде обратных положительных чисел + 1. Например, выполним конвертацию: 15 из двоичной системы счисления в двоичную, используя метод «твоих complement»:

Сначала выполним инверсию представления 1: **0000 0111**
Инверсия (обратное): **1111 1000** (инверсия представления) | 1111 1000
Затем добавляем к числу прямому: **1111 1001**

Конвертация: -15; из двоичной системы счисления в двоичную:

Представление положительного: 15: **0100 1100**
Инверсия (обратное): **1011 0011**
Добавляем к числу прямому: **1011 0100**

Почему мы добавляем прямому? Рассмотрим это на примере 15(unsigned). Если противоположительное отрицательного числа является его положительным формы, то 0 может дать представление: 0000 0000 (положительный ноль) и 1111 1111 (отрицательный ноль). При добавлении единицы, в 1111 1111 произойдет переполнение, а значение изменится на 0000 0000. Таким образом, мы получим 0 и для положительного, и для отрицательного. Если же мы к прямой инверсии почему, необходимо добавить единицу, то переполнения не произойдет и мы получим 1 для отрицательного.

Перед тем, как конвертировать отрицательное число из двоичной системы счисления в двоичную, сначала нужно понять, что это отрицательное число, на основе бит. Если это 0, то смело используйте способы, приведенные выше, для целых чисел (unsigned). Если же значение бита является 1, то тогда нужно конвертировать как бита, затем добавить единицу. Затем конвертировать в двоичную систему, и уже после этого менять знак двоичного числа на отрицательный (потому что значений бит означалось бы отрицательным).

Например, выполним конвертацию двоичного 1001 1110 (отрицательный метод «твоих complement») в десятичную систему счисления:

Инверс: 1001 1110
Инверсия (обратное): **0110 0001**
Добавляем единицу: **0110 0010**
Конвертация в десятичную систему (знаковое): 2^5 + 2^1 = 32 + 2 = 34; 2^6 + 2^5 + 2^0 = 64 + 32 + 1 = 97; 2^6 + 2^5 + 2^4 + 2^1 = 64 + 32 + 16 + 2 = 114; 2^6 + 2^5 + 2^4 + 2^1 + 2^0 = 64 + 32 + 16 + 2 + 1 = 115.

Вот так же легко! Если бит равен 0, то мы смело пишем, что это положительное число. Если же бит равен 1, то мы конвертируем как бита, затем добавляем единицу, и уже после этого конвертируем в двоичную систему. Если же бит равен 1, то мы конвертируем как бита, затем добавляем единицу, и уже после этого конвертируем в двоичную систему. Если же бит равен 0, то мы смело пишем, что это положительное число.

Почему так важен ноль данных?

Рассмотрим двоичные числа 1011 0100. Что это за число в десятичной системе? Вы, наверное, подумаете, что это 180, а если бы это было двоичное (unsigned) 180 (unsigned). Но вы были бы правы. Однако, если здесь используется метод «твоих complement», то результат будет другой. Ну. Также значение еще выше есть дополнение значения чистых положительных.

Так как же знать C++ понимает в каком числе конвертировать, 127 (0101 в 180 или в 76)

Мы, помимо обзора мы рассмотрим того данного переменной, компилятор и процессор заботится о делении конвертации этого значения в соответствующие представления. Бит отрицательного типа данных, когда мы пишем какое значение, компилятор и процессор заботится о делении конвертации в соответствующие представления.

Тип переменной влияет на то, как компилятор будет хранить число в оборотах в машинном формы. Поэтому, если мы указали целочисленный тип данных unsigned, то компилятор знает, что 127 (0101) — это (unsigned) двоичное число, а его представление в двоичной системе отличается. Если же мы указали переменной является целочисленный тип (signed), то компилятор знает, что 1011 0100 (unsigned) с помощью метода «твоих complement» это и представления в двоичной системе счисления является число -76.

Тест

Задание №1

Конвертируйте двоичное число 0100 1101 в десятичную систему счисления.

Задание №2

Конвертируйте двоичное число 10 в 8-битное двоичное число (signed) используя метод «твоих complement».

Задание №3

Конвертируйте двоичное число 1010 0010 в десятичное (signed).

Задание №4

Конвертируйте двоичное число 1010 0010 в десятичное (signed).

Задание №5

Конвертируйте двоичное число 1010 0010 в десятичное (signed) используя метод «твоих complement».

Задание №6

Напишите программу, которая просит пользователя ввести число от 0 до 255. Выведите его как 8-битное двоичное число (в виде из нуля и единиц. Не используйте побитовые операторы.

Лайфхак:

1. Воспользуйтесь способом конвертации №2. Предположите, что наименьшее число для сравнения является 128.
2. Напишите функцию для проверки входных чисел. Если число меньше 128, или больше, рекомендуем на 2.0 и. числа: 1, 2, 4, 8, 16, 32, 64 и 128. Если или нет, то выводится 1, если нет — выводится 0.

Ответы

Ответ №1:

Двоичный символ	0	1	0	0	1	1	0	1
+ Значение символа	128	64	32	16	8	4	2	1
= Результат (77)	0	64	0	0	8	4	0	1

Ответ: 77.

Ответ №2:

Используя способ №1:

10 / 2 = 40 (0)
40 / 2 = 20 (0)
20 / 2 = 10 (0)
10 / 2 = 5 (0)
5 / 2 = 2 (1)
2 / 2 = 1 (0)
1 / 2 = 0 (1)

Остатки читаем снизу вверх и записываем в одну строку: 101 1101.

Ответ: 0101 1101.

Используя способ №2:

Наибольшее число, умноженное на 2, но которое меньше 10 — это 64.

10 >= 64? Да, 64-й бит равен 1. 10 - 64 = 20.
20 >= 32? Нет, 32-й бит равен 0.
20 >= 16? Да, 16-й бит равен 1. 20 - 16 = 32.
13 >= 8? Да, 8-й бит равен 1. 13 - 8 = 5.
5 >= 4? Да, 4-й бит равен 1. 5 - 4 = 1.
1 >= 2? Нет, 2-й бит равен 0.
1 >= 1? Да, 1-й бит равен 1.

Ответ: 0101 1101.

Ответ №3:

Мы уже знаем из предыдущего примера, что 93 — это 0101 1101
Поэтому инверсия бита: 1010 0010
И добавляем единицу: 1010 0011

Ответ: 1010 0011.

Ответ №4:

Работая справа налево:

1010 0010 = (0 * 1) + (1 * 2) + (0 * 4) + (0 * 8) + (0 * 16) + (1 * 32) + (0 * 64) + (1 * 128) = 2 + 32 + 128 = 162

Ответ: 162.

Ответ №5:

Инверс: 1010 0010
Инверсия бита: 0101 1101
Добавляем единицу: 0101 1110
Конвертация в десятичную систему счисления: 2^1 + 2^2 + 2^3 + 2^4 + 2^6 = 2 + 4 + 8 + 16 + 64 = 94

Так как здесь используется метод «твоих complement», а значение бит является отрицательным, то результат: -94

Ответ: -94.

Ответ №6:

```
// Для этого нам нужно:
// 1. Спросить у пользователя число.
// 2. Сравнить с 128 (наименьшее 8-битное). 128, 64, 32 и 2.0 и
int getNumberFromUser(int, int, int);
int main()
{
    // Переменная для хранения числа, введенного пользователем.
    int iNumber = 0;
    // Переменная для цикла.
    int iCount = 0;
    // Переменная для того, сравнивать ли 2 и значения или нет.
    int iCompare = 0;
    // Переменная для хранения ответа.
    int iAnswer = 0;

    while (1)
    {
        iCount = 0;
        iCompare = 0;
        iAnswer = 0;
        cout << "Enter an integer between 0 and 255: ";
        iNumber = getNumberFromUser();
        cout << " = ";

        // Сравниваем с 128.
        iCompare = iNumber - 128;
        if (iCompare >= 0)
        {
            iAnswer = iAnswer + 128;
            iNumber = iNumber - 128;
        }
        // Сравниваем с 64.
        iCompare = iNumber - 64;
        if (iCompare >= 0)
        {
            iAnswer = iAnswer + 64;
            iNumber = iNumber - 64;
        }
        // Сравниваем с 32.
        iCompare = iNumber - 32;
        if (iCompare >= 0)
        {
            iAnswer = iAnswer + 32;
            iNumber = iNumber - 32;
        }
        // Сравниваем с 16.
        iCompare = iNumber - 16;
        if (iCompare >= 0)
        {
            iAnswer = iAnswer + 16;
            iNumber = iNumber - 16;
        }
        // Сравниваем с 8.
        iCompare = iNumber - 8;
        if (iCompare >= 0)
        {
            iAnswer = iAnswer + 8;
            iNumber = iNumber - 8;
        }
        // Сравниваем с 4.
        iCompare = iNumber - 4;
        if (iCompare >= 0)
        {
            iAnswer = iAnswer + 4;
            iNumber = iNumber - 4;
        }
        // Сравниваем с 2.
        iCompare = iNumber - 2;
        if (iCompare >= 0)
        {
            iAnswer = iAnswer + 2;
            iNumber = iNumber - 2;
        }
        // Сравниваем с 1.
        iCompare = iNumber - 1;
        if (iCompare >= 0)
        {
            iAnswer = iAnswer + 1;
            iNumber = iNumber - 1;
        }
        // Выводим ответ.
        cout << iAnswer << "\n";
        // Если пользователь ввел 0, то выходим из цикла.
        if (iAnswer == 0)
            break;
    }
    return 0;
}
```